

ARREGLOS UNIDIMENSIONALES (ARRAYS) – VECTORES –

Un arreglo ,en programación, es un conjunto de variables o elementos agrupados en forma continua bajo un nombre, cuyo contenido pertenece a un tipo de dato homogéneo.

En realidad, son un conjunto de direcciones de memoria continuas que se reservan para que cada una almacene un contenido del mismo tipo de dato (declarado en con el nombre del vector) que están referenciadas bajo un nombre.

Una definición más representativa puede ser: Un vector o arreglo, es una variable muy grande que contiene un conjunto de variables pequeñas, en la cual cada variable pequeña pertenece al mismo tipo de dato que la variable grande (tipo de dato homogéneo).

Tipos de Arrays: Unidimensionales (Vectores), Bidimensionales (Matices), Multidimensionales.

Declaración de un vector

Tipo de dato **Identificador o nombre** **[tamaño o dimensión];**

Tipo de dato: int, float, char.

Identificador o nombre: Es la identificación del arreglo en el programa, tiene las mismas restricciones para asignar el nombre a una variable (Es la referencia o puntero de la zona de la zona de memoria reservada).

Un puntero es una variable cuyo contenido, es una dirección de memoria (representada en Base 16 – Hexadecimal]], es decir que apunta a una dirección de memoria, por esto se lo conceptualiza “puntero”.

Tamaño o dimensión: Es la cantidad de direcciones de memoria que se van a reservar o la cantidad de variables pequeñas que va a contener la variable grande (Identificador o nombre), siempre es un int >0 y menor a un int que no supere la memoria del programa.

Nosotros vamos a trabajar con vectores estáticos en tiempo de compilación, es decir que antes de compilar se debe conocer el tamaño del vector (a través de una constante, una variable definida previamente al vector con un valor de inicialización, etc.).

Ejemplo: Caso correcto.

```
#define TAM 5
```

```
int main ()
```

```
{
```

```
int dimension = 4;
```

```
int vector [TAM], vector2[dimension];
```

```
..
```

```
}
```

Ejemplo: Caso incorrecto.

```
int main()
```

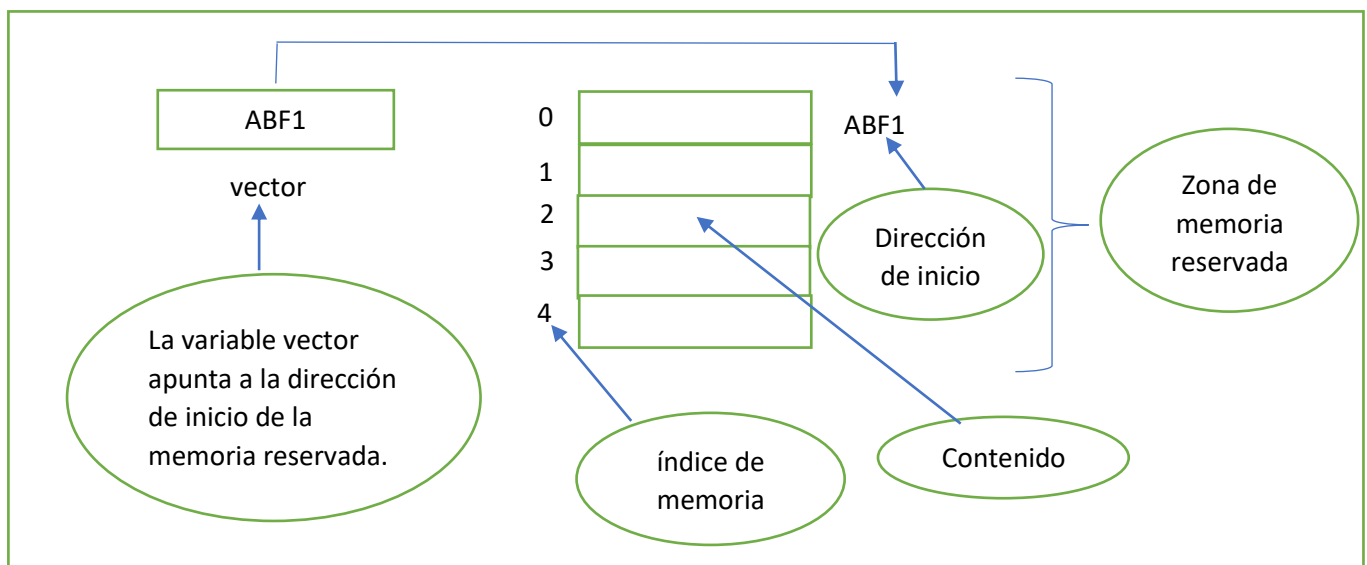
```
{ int a, vector[a]; //Error, porque la variable a no tiene valor asignado por el programa.
```

```
...
```

```
}
```

Representación en la memoria Principal

```
int vector[5];
```



Los vectores comienzan desde la posición cero, ya que es la misma posición de la dirección de memoria a la que apuntan (dirección de inicio).

Acceso

“Solo se puede acceder al contenido del vector”, se realiza a través del índice, el mismo es un número int comprendido entre 0 y el tamaño -1 del vector.

$0 \leq \text{índice} \leq (\text{Tamaño del vector} - 1)$ o $0 \leq \text{índice} < \text{TAM}$

Por ejemplo

int vector[4]; //El índice estará comprendido entre 0 y 3.

int a=5,b=3,c=0;

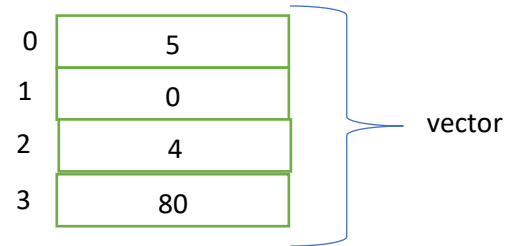
/*Cómo índice se puede colocar una expresión: constante int, operación int, variable int, función que retorne un int o cualquier combinación mientras esté comprendido en el rango válido no hay inconveniente, pero esto no es validado por el lenguaje, lo debe realizar el programador*/.

vector[0] = 5 ; //índice = 0, contenido=5.

vector[a-4]= 0; //índice = 1, contenido=0.

vector[(b+c) - 1] = 4; //índice = 2, contenido=4.

vector[es_par(5) + b] = 80; //índice = 3, contenido=80.



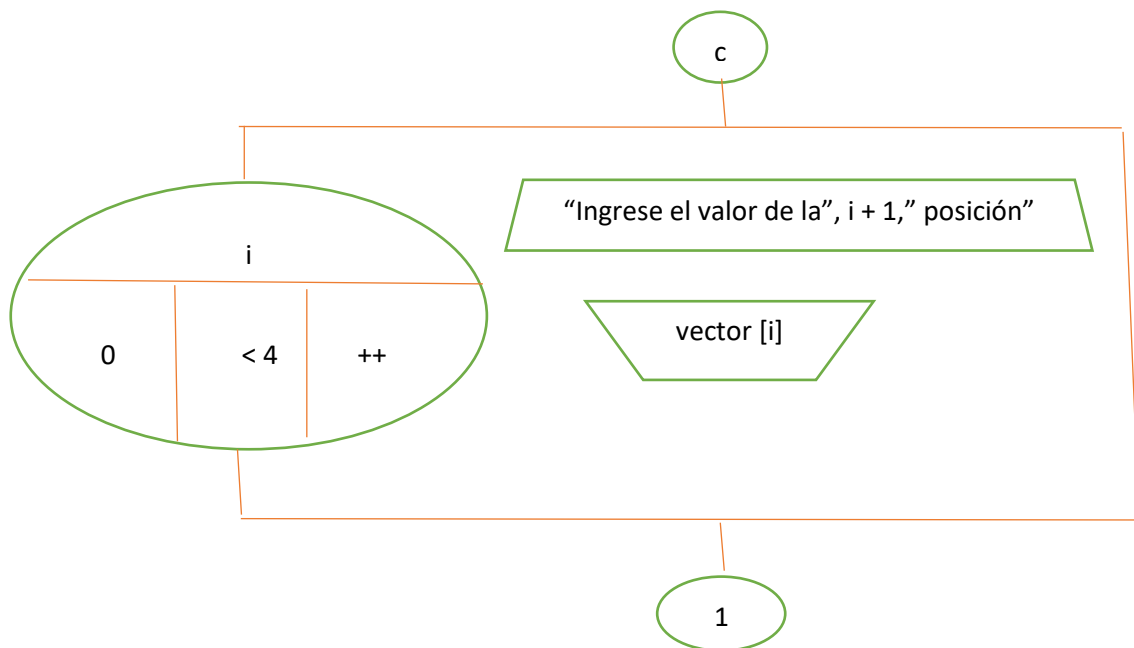
Retorna 1 si el argumento es par
y cero en caso contrario.

A modo general el acceso a un arreglo es **Identificador[índice]**

Recorrido

Por lo general se realiza con la estructura de iteración (definida o condicionada), ciclos for, while, do-while.

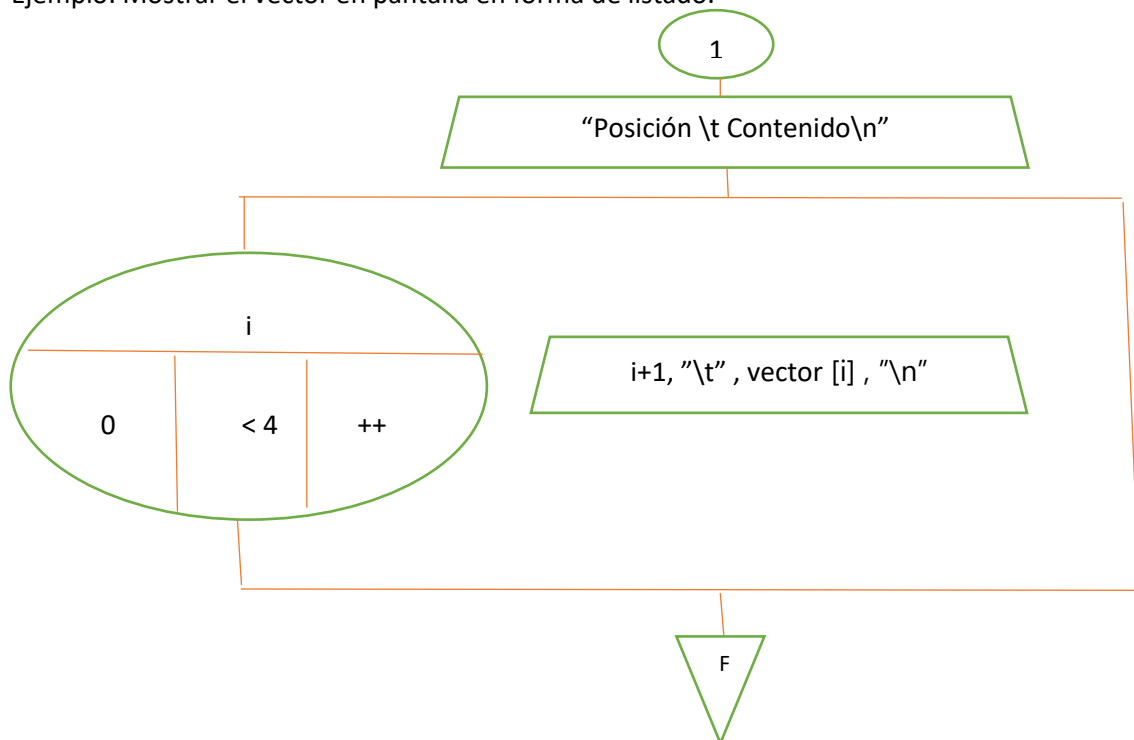
Por ejemplo: Para solicitar al usuario el ingreso de los 4 valores al vector.



Codificación

```
int main ()
{
    int vector [4], i;
    for(i=0;i<4;i++)
    {
        /*Para el usuario el índice 0 del vector es la posición 1, índice 1 la
        posición 2,.....*/
        printf("Ingrese el valor de la %d posición : ", i+1);
        scanf("%d",&vector[i]);//ESCRITURA DEL VECTOR EN LA POSICIÓN i.
    }
}
```

Ejemplo: Mostrar el vector en pantalla en forma de listado.



```
printf("\nPosición \t Contenido\n ");

for(i=0;i<4;i++)

{

    printf("%2d %4d\n",i+1,vector[i]); //LECTURA DEL VECTOR EN LA POSICIÓN i.

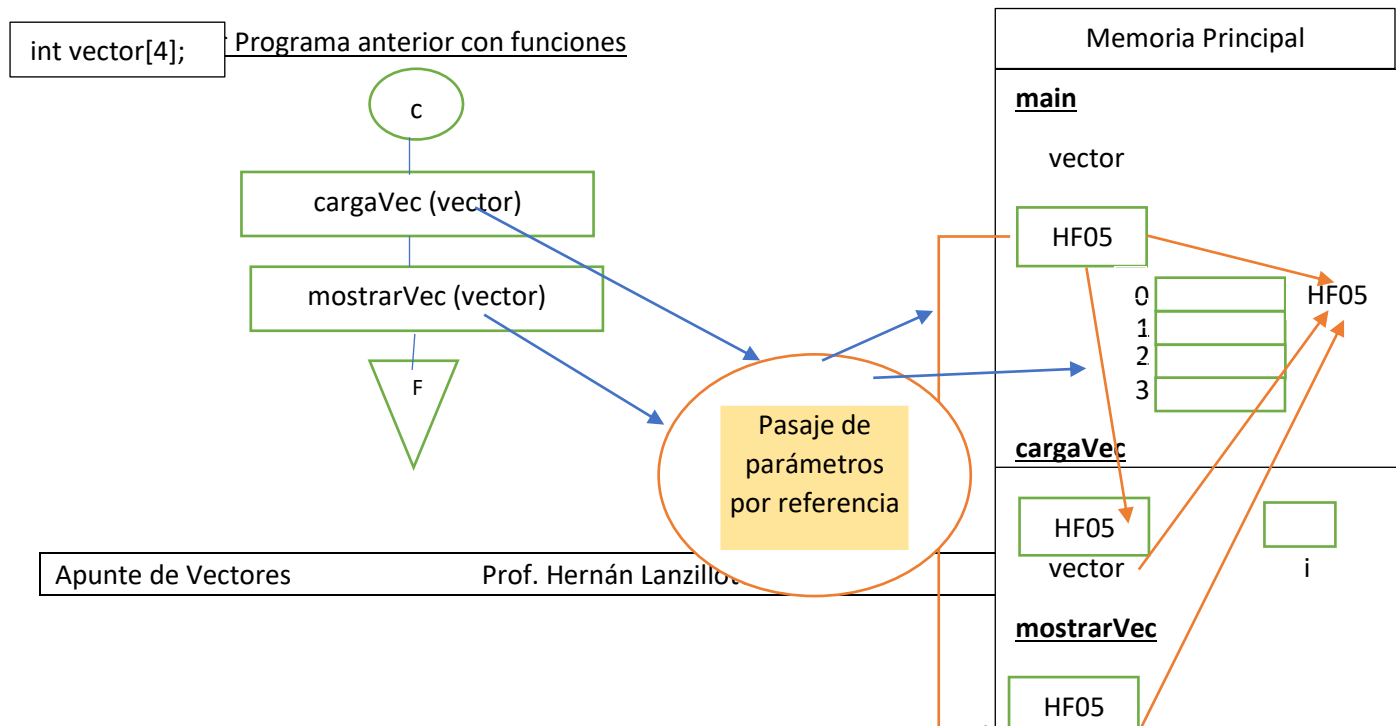
}

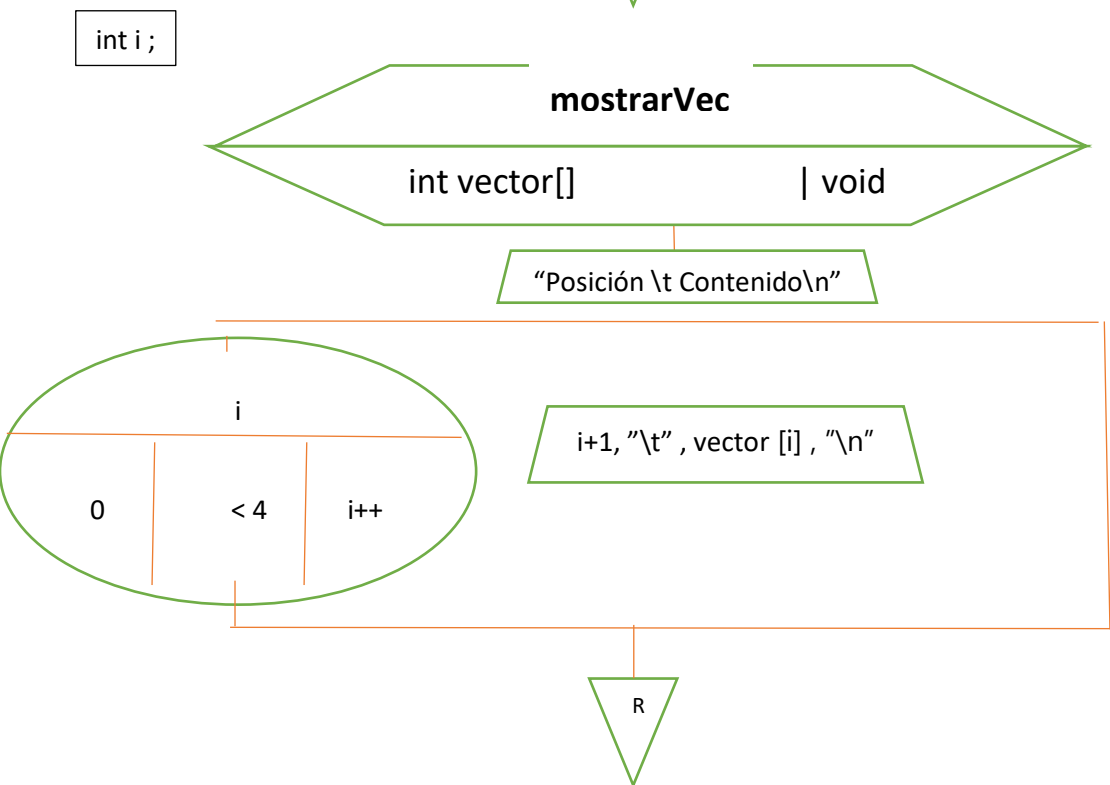
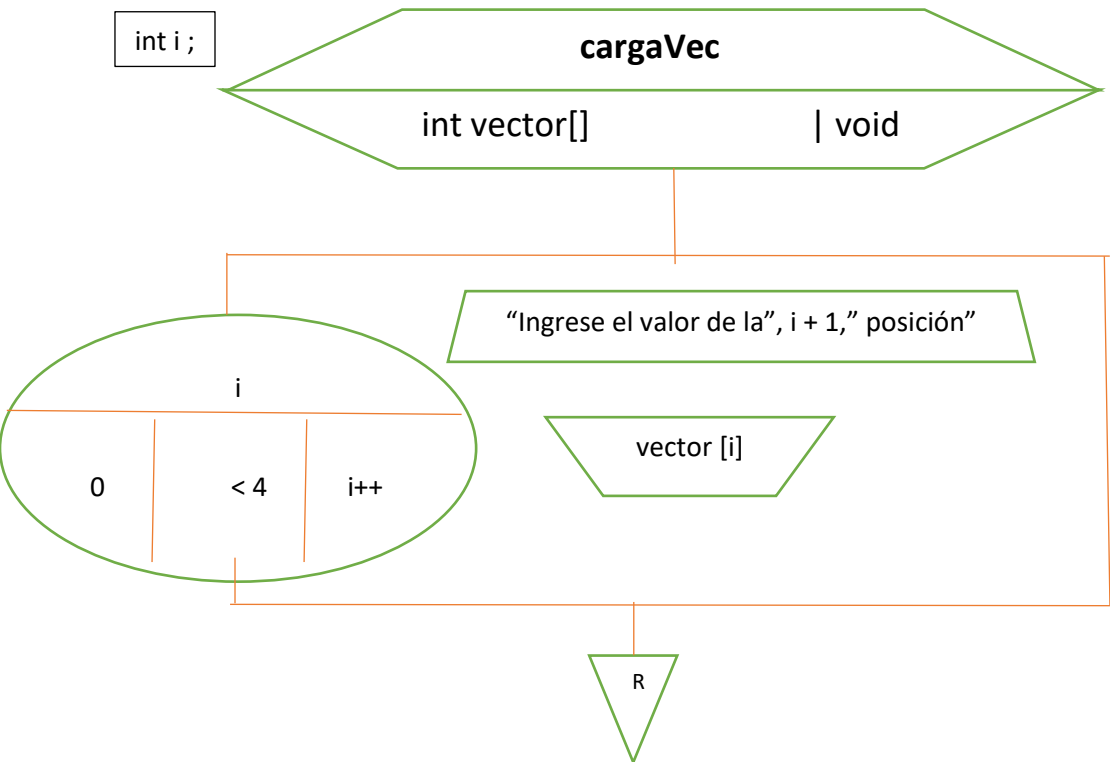
return 0;

} //fin main
```

Un arreglo se puede enviar a una función como parámetro actual, real o argumento tan solo con colocar su nombre, ya que el mismo es una referencia o puntero a la dirección de inicio a la que se debe apuntar (zona de memoria reservada). **En este caso el pasaje de parámetros es por referencia, no por valor, porque se envía una dirección.** En este caso solo se coloca el nombre de arreglo. Si lo que pretende es enviar un sólo elemento del vector, se coloca el identificador [índice]

Como prototipo se coloca el tipo de dato del arreglo seguido de []





Codificación

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <locale.h>
```

```
#define TAM 4 //CANTIDAD DE ELEMENTOS DEL VECTOR
```

```
//PROTOTIPOS DE FUNCIONES
```

```
void cargaVec (int []);
```

```
void mostrarVec (int []);
```

```
//PROGRAMA PRINCIPAL.
```

```
int main ()
```

```
{
```

```
int vector[TAM];
```

```
setlocale(LC_ALL,"spanish");//IDIOMA ESPAÑOL.
```

```
printf("\nmain: Dirección de inicio (referencia) a la que apunta el vector %x\n",vector);
```

```
cargaVec(vector); //SÓLO SE COLOCA EL NOMBRE DEL ARREGLO COMO PARÁMETRO.
```

```
mostrarVec(vector); //SÓLO SE COLOCA EL NOMBRE DEL ARREGLO COMO PARÁMETRO.
```

```
system("pause");
```

```
return 0;
```

```
}
```

```
//DEFINICIÓN DE FUNCIONES.
```

```
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

```
void cargaVec(int vector[] )
```

```
{ int i;
```

```
printf("\nFunción cargaVec: Dirección a la que apunta el vector %x\n",vector);
```

```
for( i = 0; i < TAM; i++)
```

```
{
```

```
printf("Ingrese el valor de la %d posición : ",i+1);
```

```
scanf("%d",&vector[i]);//ESCRITURA EN EL VECTOR EN LA POSICIÓN i.
```

```
}
```

```
}
```

```
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

```
void mostrarVec(int vector[] )
```

```
{ int i;
```

```
printf("\nFunción mostrarVec: Dirección a la que apunta el vector %x\n",vector);
```

```
printf("\nPosición Contenido\n ");
```

```
for(i=0;i< TAM ;i++)
```

```
{
```

```
printf("%2d \t %2d\n",i+1,vector[i]);//LECTURA DEL VECTOR EN LA POSICIÓN i.
```

```
}
```

```
}
```